PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies

Student Name	GORRELA LALITHA VENKATA DURGA
Roll Number	26000920
Branch	CSM
Section	2
Selected Case Studies	10

Prepared according to the activity sheet deliverables: problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output samples, and conclusion.

Note: The PDF uses the ten topic names supplied in the request. Sample data, rates, and policies are illustrative and should be changed if the faculty provides different rules.

1. University Scholarship Allocation

Problem Statement

Develop a program that checks student applications using aggregate marks, attendance, and family income. Eligible applicants are ranked by marks, with attendance used to break ties.

Objectives

• Check each applicant against the stated eligibility rules.

• Rank eligible applicants consistently.

• Display selected and rejected applicants with reasons.

Assumptions

• Marks are out of 100; attendance is a percentage.

• Eligibility: marks >= 75, attendance >= 80%, annual family income <= ■250000.

• Higher marks rank first; attendance breaks equal-mark ties.

Algorithm

1. Read applicant records.

2. Check marks, attendance, and income conditions.

3. Store eligible applicants in a list.

4. Sort by marks and then attendance in descending order.

5. Print ranking and eligibility results.

Flowchart

START → **Read applicant records** → **Check eligibility** → **Any eligible applicants?** → **Sort eligible list** → **Displ**

Pseudocode

```
START\nINPUT applicants\neligible = empty list\nFOR each applicant\n  IF marks >= 75 AND attendance >= 80 AND income <= 250000
```

Python Source Code

```
applicants = [
    ("Asha", 88, 91, 180000),
    ("Balu", 78, 85, 220000),
    ("Charan", 72, 90, 150000),
    ("Divya", 88, 95, 240000)
]
eligible = []
for name, marks, attendance, income in applicants:
    if marks >= 75 and attendance >= 80 and income <= 250000:
        eligible.append((name, marks, attendance, income))
eligible.sort(key=lambda x: (x[1], x[2]), reverse=True)
print("Eligible scholarship applicants:")
for rank, item in enumerate(eligible, 1):
    print(rank, item[0], "Marks:", item[1],
          "Attendance:", str(item[2]) + "%")
```

Test Cases

Input / Scenario	Expected result
Asha 88, 91%, ■180000	Eligible
Charan 72, 90%, ■150000	Rejected: marks below 75
Divya 88, 95%, ■240000	Rank 1

Output Screenshot (sample console output)

Conclusion

The program applies the stated eligibility rules and ranks qualified applicants transparently. Actual scholarship decisions should use the institution’s official policy.

2. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Compare mobile plans for a customer using monthly data and call usage. Calculate the total charge for each plan and select the least expensive plan that satisfies the customer's usage needs and budget.

Objectives

• Calculate plan costs including excess-use charges.

• Reject plans that cannot meet usage or budget requirements.

• Recommend the lowest-cost feasible plan.

Assumptions

• All prices are illustrative monthly amounts in rupees.

• A plan is feasible only if its data and call limits cover the usage.

• Excess data costs ■20 per GB and excess calls cost ■1 per minute; budget is ■600.

Algorithm

1. Enter customer usage and plan details.

2. For each plan, calculate base price plus excess charges.

3. Check data, calls, and budget constraints.

4. Keep feasible plans and identify the minimum-cost option.

5. Display plan costs and recommendation.

Flowchart

START → Read usage and plans → Calculate each plan cost → Check limits and budget → Select minimum feasibl

Pseudocode

```
START\nINPUT data_used, call_minutes, budget\nFOR each plan\n  cost = monthly_fee\n  IF usage exceeds allowance, add excess ch
```

Python Source Code

```
plans = [
    ("Basic", 299, 5, 300),
    ("Smart", 449, 10, 600),
    ("Max", 599, 20, 1000)
]
data_used = 8
call_minutes = 500
budget = 600
feasible = []
for name, fee, data_limit, call_limit in plans:
    extra_data = max(0, data_used - data_limit)
    extra_calls = max(0, call_minutes - call_limit)
    cost = fee + extra_data * 20 + extra_calls * 1
    meets_usage = data_limit >= data_used and call_limit >= call_minutes
    print(name, "cost = Rs.", cost)
    if meets_usage and cost <= budget:
        feasible.append((cost, name))
if feasible:
    best = min(feasible)
    print("Recommended plan:", best[1], "Rs.", best[0])
else:
    print("No plan meets usage and budget requirements.")
```

Test Cases

Input / Scenario	Expected result
8 GB, 500 min, ■600 Smart plan provides 10 GB and 600 minutes	Feasible at ■449
8 GB, 500 min, ■600 Basic plan limits are too low	Not feasible

Output Screenshot (sample console output)

Conclusion

The comparison supports a cost-based choice while checking service limits and budget. The sample plans and rates are hypothetical.

3. Campus Event Financial Planning

Problem Statement

Prepare a financial plan for a campus event by adding venue, equipment, catering, transport, and certificate expenses. Calculate sponsorship, participant contribution, and the number of registrations needed to break even.

Objectives

• Calculate total event expenses.

• Find the funding gap after sponsorship.

• Calculate contribution per participant and break-even registration count.

Assumptions

• Venue ■12000, equipment ■5000, catering ■180 per participant, transport ■3000, certificates ■2000.

• Sponsorship is ■10000 and participant fee is ■250.

• Break-even means total income is at least total cost.

Algorithm

1. Set fixed and per-participant costs.

2. Calculate total cost for the expected participants.

3. Subtract sponsorship from the total cost.

4. Calculate the minimum registrations needed using participant fee minus variable cost per participant.

5. Display financial summary.

Flowchart

START → Enter costs, sponsorship and fee → Calculate total cost → Calculate break-even count → Compare exp

Pseudocode

```
START\nINPUT expected participants\nfixed_cost = venue + equipment + transport + certificates\nper_person_cost = catering cost
```

Python Source Code

```
from math import ceil
venue = 12000
equipment = 5000
transport = 3000
certificates = 2000
catering_per_person = 180
sponsorship = 10000
fee_per_person = 250
expected = 100
fixed_cost = venue + equipment + transport + certificates
total_cost = fixed_cost + catering_per_person * expected
total_income = sponsorship + fee_per_person * expected
contribution = fee_per_person - catering_per_person
break_even = ceil((fixed_cost - sponsorship) / contribution)
print("Total cost: Rs.", total_cost)
print("Expected income: Rs.", total_income)
print("Surplus/shortfall: Rs.", total_income - total_cost)
print("Break-even registrations:", break_even)
```

Test Cases

Input / Scenario	Expected result
100 participants Cost ■40000; income ■35000	Shortfall ■5000
Break-even Fixed cost ■22000; sponsorship ■10000; contribution ■70	172 registrations

Output Screenshot (sample console output)

Conclusion

The program estimates whether the event budget is viable and shows the registration target required under the stated assumptions.

4. Examination Result and Grace-Mark Validation

Problem Statement

Process a student's marks in several subjects, apply subject-wise pass marks and a limited grace-mark policy, and ensure final marks never exceed the maximum mark.

Objectives

• Determine pass or fail subject-wise.

• Apply grace marks only within the specified policy.

• Prevent final marks from exceeding 100.

Assumptions

• Each subject is out of 100 and requires 35 to pass.

• At most 3 grace marks may be added to a subject that is short by 1–3 marks.

• Grace marks are used only once in total; subjects are processed in order.

Algorithm

1. Read subject marks.

2. For each failed subject, calculate the shortfall.

3. If the shortfall is 1–3 and grace remains, add the required grace marks.

4. Cap marks at 100 and record adjusted marks.

5. Print subject results and overall status.

Flowchart

START → Read marks → Check each subject → Eligible for grace and grace unused? → Adjust if allowed → Deter

Pseudocode

```
START\nINPUT marks list\ngrace_used = 0\nFOR each subject mark\n  IF mark < 35 AND shortfall <= 3 AND grace_used == 0\n    mar
```

Python Source Code

```
marks = [67, 33, 81, 35]
pass_mark = 35
grace_limit = 3
grace_used = False
final_marks = []
for mark in marks:
    if mark < pass_mark and not grace_used:
        shortfall = pass_mark - mark
        if shortfall <= grace_limit:
            mark = min(100, mark + shortfall)
            grace_used = True
    final_marks.append(mark)
for i, mark in enumerate(final_marks, 1):
    status = "PASS" if mark >= pass_mark else "FAIL"
    print("Subject", i, ":", mark, status)
overall = "PASS" if all(m >= pass_mark for m in final_marks) else "FAIL"
print("Overall result:", overall)
```

Test Cases

Input / Scenario	Expected result
33/100 Shortfall 2; grace available	Adjusted to 35, PASS
30/100 Shortfall 5	No grace; FAIL
100/100 At maximum	Remains 100

Output Screenshot (sample console output)

Conclusion

The program demonstrates a controlled grace-mark rule and validates maximum marks. Institutions must substitute their officially approved rules.

5. Smart Parking Fee Management

Problem Statement

Calculate parking charges from entry and exit times, taking vehicle category and the number of hours parked into account. Validate time inputs and apply a daily maximum charge.

Objectives

• Calculate parking duration and fee.

• Apply different hourly rates for vehicle types.

• Validate times and enforce the daily cap.

Assumptions

• Times are entered as 24-hour HH:MM on the same day.

• Car rate is ■30/hour and bike rate is ■15/hour.

• Any partial hour is charged as a full hour; maximum charge is ■300 per day.

Algorithm

1. Read vehicle type, entry time, and exit time.

2. Validate HH:MM values and ensure exit is after entry.

3. Calculate duration in minutes and round up to hours.

4. Multiply by the vehicle rate and apply the daily cap.

5. Display duration and fee.

Flowchart

START → Read vehicle and times → Validate inputs → Calculate duration → Round up hours → Apply rate and ca

Pseudocode

```
START\nINPUT vehicle, entry, exit\nIF invalid time or exit <= entry, DISPLAY error\nELSE minutes = exit - entry\n hours = ceil
```

Python Source Code

```
from math import ceil
def to_minutes(value):
    parts = value.split(":")
    if len(parts) != 2 or not all(p.isdigit() for p in parts):
        return -1
    h, m = int(parts[0]), int(parts[1])
    if h < 0 or h > 23 or m < 0 or m > 59:
        return -1
    return h * 60 + m

vehicle = "car"
entry = "09:15"
exit_time = "13:40"
start = to_minutes(entry)
end = to_minutes(exit_time)
if start < 0 or end < 0 or end <= start:
    print("Invalid parking time.")
else:
    hours = ceil((end - start) / 60)
    rate = 30 if vehicle == "car" else 15
    fee = min(hours * rate, 300)
    print("Vehicle:", vehicle)
    print("Charged hours:", hours)
    print("Parking fee: Rs.", fee)
```

Test Cases

Input / Scenario	Expected result
Car 09:15–13:40 4 hours 25 minutes	5 hours; ■150
Bike 10:00–11:00 1 hour	■15

Input / Scenario	Expected result
Exit before entry Invalid duration	Error message

Output Screenshot (sample console output)

Conclusion

The solution calculates parking charges with time validation, category rates, and a cap. Overnight parking would require an extended date-aware rule.

6. Fuel Consumption and Route Comparison

Problem Statement

Compare different vehicle-and-route options using distance, mileage, fuel price, and a required fuel reserve. Select the lowest-cost option that has enough fuel capacity for the journey and reserve.

Objectives

• Calculate fuel needed and journey cost.

• Check whether each option meets reserve requirements.

• Identify the least-cost feasible option.

Assumptions

• Fuel required = distance / mileage.

• A 10% reserve is required in addition to journey fuel.

• Fuel price is ■105 per litre; each option provides its available tank fuel.

Algorithm

1. Read distance, mileage, and available fuel for each option.

2. Calculate journey fuel and required reserve-inclusive fuel.

3. Calculate journey cost.

4. Mark options feasible if available fuel meets the requirement.

5. Select the lowest-cost feasible option.

Flowchart

START → Read route/vehicle data → Calculate fuel + reserve → Check capacity → Compare feasible costs → Dis

Pseudocode

```
START\nINPUT options\nFOR each option\n  journey_fuel = distance / mileage\n  required_fuel = journey_fuel * 1.10\n  cost = jo
```

Python Source Code

```
fuel_price = 105
options = [
    ("Car A", 180, 15, 15),
    ("Car B", 180, 20, 12),
    ("Car C", 160, 16, 12)
]
feasible = []
for name, distance, mileage, available_fuel in options:
    journey_fuel = distance / mileage
    required_fuel = journey_fuel * 1.10
    cost = journey_fuel * fuel_price
    print(name, "fuel:", round(journey_fuel, 2),
          "L; cost: Rs.", round(cost, 2))
    if available_fuel >= required_fuel:
        feasible.append((cost, name))
if feasible:
    best = min(feasible)
    print("Lowest-cost feasible option:", best[1])
else:
    print("No option meets the reserve requirement.")
```

Test Cases

Input / Scenario	Expected result
Car A 180 km / 15 km per L = 12 L; reserve total 13.2 L	Feasible with 15 L
Car B 9 L; reserve total 9.9 L	Feasible with 12 L
Car B vs Car A ■945 vs ■1260	Car B is cheaper

Output Screenshot (sample console output)

Conclusion

The program compares costs while respecting a fuel-reserve constraint. Real results depend on actual mileage, fuel price, and route conditions.

7. Progressive Water Tariff Analysis

Problem Statement

Calculate household water bills using progressive consumption slabs, a fixed fee, and a penalty for consumption above a defined threshold. Compare household bills.

Objectives

• Apply different rates to each consumption slab.

• Add fixed charges and any applicable penalty.

• Compare total bills and identify the highest bill.

Assumptions

• First 10 kL cost ■5/kL; next 10 kL cost ■8/kL; consumption above 20 kL costs ■12/kL.

• Fixed fee is ■40 per household.

• A ■100 penalty applies when usage exceeds 30 kL.

Algorithm

1. Read each household's consumption.

2. Calculate charges slab by slab.

3. Add fixed fee and any penalty.

4. Store and print each bill.

5. Identify the household with the highest bill.

Flowchart

START → Read consumption list → Calculate slab charges → Add fixed fee/penalty → Compare bills → Display b

Pseudocode

```
START\nFOR each household\n  charge = 0\n  charge first 10 kL at rate 5\n  charge next 10 kL at rate 8\n  charge remaining kL
```

Python Source Code

```
def water_bill(units):
    first = min(units, 10) * 5
    second = min(max(units - 10, 0), 10) * 8
    third = max(units - 20, 0) * 12
    penalty = 100 if units > 30 else 0
    return first + second + third + 40 + penalty

households = [("H1", 8), ("H2", 18), ("H3", 35)]
bills = []
for name, units in households:
    bill = water_bill(units)
    bills.append((name, bill))
    print(name, units, "kL: Rs.", bill)
highest = max(bills, key=lambda item: item[1])
print("Highest bill:", highest[0], "Rs.", highest[1])
```

Test Cases

Input / Scenario	Expected result
8 kL 8×■5 + ■40	■80
18 kL ■50 + ■64 + ■40	■154
35 kL ■50 + ■80 + ■180 + ■40 + ■100	■450

Output Screenshot (sample console output)

Conclusion

The program demonstrates progressive billing and flags high usage through a penalty. Rates and thresholds are sample values.

8. Text Compression Using Character Runs

Problem Statement

Compress a string by replacing each consecutive run of identical characters with the character followed by its count. Compare the original and encoded lengths and keep the original if compression does not reduce length.

Objectives

• Group consecutive repeated characters.

• Encode each run with its character and count.

• Compare lengths and select the shorter representation.

Assumptions

• Input contains ordinary text without a special escape format.

• Each run is encoded as character plus decimal count, including count 1.

• If the encoded string is not shorter, the original string is retained.

Algorithm

1. Read a string.

2. If empty, report empty input.

3. Scan characters and count each consecutive run.

4. Append character and run count to encoded output.

5. Compare lengths and display the selected result.

Flowchart

START → Read text → Empty? → Scan consecutive runs → Build encoding → Compare lengths → Display selected

Pseudocode

```
START\nINPUT text\nIF text is empty, DISPLAY empty\nELSE\n  encoded = empty string\n  count consecutive equal characters\n  ap
```

Python Source Code

```
def run_encode(text):
    if not text:
        return ""
    encoded = ""
    count = 1
    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            encoded += text[i - 1] + str(count)
            count = 1
    encoded += text[-1] + str(count)
    return encoded

original = "aaabbccccccccd"
encoded = run_encode(original)
chosen = encoded if len(encoded) < len(original) else original
print("Original:", original)
print("Encoded:", encoded)
print("Original length:", len(original))
print("Encoded length:", len(encoded))
print("Selected representation:", chosen)
```

Test Cases

Input / Scenario	Expected result
aaabbccccccccd Runs: a×3, b×2, c×8, d×1	a3b2c8d1; shorter
abcd All characters occur once	Original retained

Input / Scenario	Expected result
Empty string No characters	Empty output

Output Screenshot (sample console output)

Conclusion

Run-length encoding is effective when many adjacent characters repeat. A production format would need escaping or a clear rule for digits in the original text.

9. Integrated E-Commerce Order Processing and Reconciliation

Problem Statement

Process product orders with quantities, discounts, dispatches, returns, and stock updates. Calculate the final revenue and detect any difference between ordered, dispatched, and returned quantities.

Objectives

• Calculate order totals and discounts.

• Update stock after dispatches and returns.

• Reconcile quantities and calculate net revenue.

Assumptions

• Prices and quantities are hypothetical.

• A 10% discount applies when an order subtotal is at least ■1000.

• Net sold quantity = dispatched quantity – returned quantity; stock decreases by net sold quantity.

Algorithm

1. Read product price, stock, order quantity, dispatched quantity, and returns.

2. Validate quantities against business rules.

3. Calculate subtotal and eligible discount.

4. Calculate net sold quantity and updated stock.

5. Report revenue and any quantity mismatch.

Flowchart

START → **Read order/stock data** → **Validate quantities** → **Calculate subtotal/discount** → **Update stock** → **Reconci**

Pseudocode

```
START\nINPUT price, stock, ordered, dispatched, returned\nIF invalid quantities, report error\nELSE\n  subtotal = price * orde
```

Python Source Code

```
product = "Notebook"
price = 120
stock = 50
ordered = 12
dispatched = 10
returned = 1
if returned > dispatched or dispatched > ordered:
    print("Invalid order reconciliation.")
else:
    subtotal = price * ordered
    discount = subtotal * 0.10 if subtotal >= 1000 else 0
    net_sold = dispatched - returned
    updated_stock = stock - net_sold
    revenue = price * net_sold - discount
    print("Product:", product)
    print("Order subtotal: Rs.", subtotal)
    print("Discount: Rs.", round(discount, 2))
    print("Net sold:", net_sold)
    print("Updated stock:", updated_stock)
    print("Net revenue: Rs.", round(revenue, 2))
    if dispatched != ordered:
        print("Reconciliation note: not all ordered units dispatched.")
```

Test Cases

Input / Scenario	Expected result
Ordered 12, dispatched 10, returned 1 Net sold 9	Updated stock 41
Subtotal ■1440 10% discount	■144 discount

Input / Scenario	Expected result
Returned > dispatched Invalid quantity relation	Reject record

Output Screenshot (sample console output)

Conclusion

The program brings order, dispatch, return, stock, and revenue calculations together. In a full system, discounts would normally be reconciled against the exact units invoiced.

10. Tuple-Based Transaction Audit

Problem Statement

Audit transaction records stored as tuples containing transaction ID, amount, and transaction type. Identify duplicate IDs, invalid amounts, and unusual consecutive transactions.

Objectives

• Use tuples to store transaction records.

• Detect duplicate transaction identifiers and invalid amounts.

• Summarize deposits and withdrawals and flag suspicious sequences.

Assumptions

• Each tuple is (transaction_id, amount, type).

• Amounts must be positive; type must be DEPOSIT or WITHDRAWAL.

• Three or more consecutive withdrawals are flagged for manual review, not automatically treated as fraud.

Algorithm

1. Read or define transaction tuples.

2. Track seen transaction IDs.

3. Flag duplicate IDs, non-positive amounts, and invalid types.

4. Calculate deposit and withdrawal totals for valid records.

5. Flag three consecutive withdrawals and print the audit report.

Flowchart

START → Read transaction tuples → Check ID/amount/type → Track totals and sequence → Flag issues → Displa

Pseudocode

```
START\nINPUT transactions\nseen = empty list; totals = 0\nFOR each transaction tuple\n  unpack id, amount, type\n  flag duplic
```

Python Source Code

```
transactions = [
    ("T01", 500, "DEPOSIT"),
    ("T02", 200, "WITHDRAWAL"),
    ("T02", 100, "WITHDRAWAL"),
    ("T04", -50, "DEPOSIT"),
    ("T05", 80, "WITHDRAWAL"),
    ("T06", 60, "WITHDRAWAL")
]
seen = []
deposit_total = 0
withdrawal_total = 0
withdrawal_streak = 0
issues = []
for tid, amount, kind in transactions:
    if tid in seen:
        issues.append(tid + ": duplicate ID")
    else:
        seen.append(tid)
    if amount <= 0:
        issues.append(tid + ": invalid amount")
        withdrawal_streak = 0
        continue
    if kind == "DEPOSIT":
        deposit_total += amount
        withdrawal_streak = 0
    elif kind == "WITHDRAWAL":
        withdrawal_total += amount
        withdrawal_streak += 1
        if withdrawal_streak == 3:
            issues.append(tid + ": 3 consecutive withdrawals")
    else:
        issues.append(tid + ": invalid transaction type")
        withdrawal_streak = 0
print("Deposit total: Rs.", deposit_total)
print("Withdrawal total: Rs.", withdrawal_total)
print("Audit issues:")
for issue in issues:
```

```
    print("-", issue)
```

Test Cases

Input / Scenario	Expected result
Duplicate T02 ID appears twice	Flag duplicate
Amount -50 Amount <= 0	Flag invalid amount
Unknown type Type not in allowed list	Flag invalid type

Output Screenshot (sample console output)

Conclusion

Tuple unpacking makes transaction records easy to process. Audit flags are indicators for review and do not establish wrongdoing.